

TABLE OF CONTENTS

[Public Vehicle Search](#)

[User Login](#)

[Add Vehicle](#)

[Add Parts Order](#)

[Sell Vehicle](#)

[View Vehicle Detail](#)

[Parts Statistics](#)

[Price Per Conditions](#)

[Monthly Sales Summary](#)

[Monthly Sales Drilldown](#)

[Seller History](#)

[Average Time in Inventory](#)

[Search/Add Vendor](#)

[Update Part Status](#)

Public Vehicle Search

Abstract Code

- User navigates to the **Search Vehicle form**.
 - Display the total number of vehicles currently available for sale (no pending parts).

```
SELECT
    COUNT(DISTINCT v.VIN) AS VehiclesForSale
FROM cs6400_sp26_team040.Vehicle v
LEFT JOIN cs6400_sp26_team040.Sale s
    ON v.VIN = s.VIN
LEFT JOIN cs6400_sp26_team040.Part p
    ON v.VIN = p.VIN AND p.Status != 'Installed'
WHERE s.VIN IS NULL AND
    p.VIN IS NULL;
```

- If *User Role* (**\$UserRole**) = Acquisition Specialist, Operating Manager, or Owner:
 - Also display the count of vehicles with pending parts.

```
SELECT
    COUNT(DISTINCT p.VIN) AS VehiclesWithPendingParts
FROM cs6400_sp26_team040.Part p
WHERE p.Status != 'Installed';
```

- Display search input fields: vehicle type, manufacturer, year, fuel type, drivetrain layout, color (dropdowns), keyword (free text).
- If **\$UserRole** = Acquisition Specialist, Sales Agent, Operating Manager, or Owner:
 - Also display a VIN input field.
- If **\$UserRole** = Operating Manager or Owner:
 - Also display a status filter dropdown: Sold / Unsold / All.
- When the Search button is clicked, query the database for vehicles matching the entered criteria according to the following rules:
 - If **\$UserRole** = Public User or Sales Agent: return only unsold vehicles with no pending parts.
 - If **\$UserRole** = Acquisition Specialist: return all unsold vehicles regardless of parts status.
 - If **\$UserRole** = Operating Manager or Owner: return vehicles matching the selected status filter (Sold / Unsold / All); when Unsold or All is selected, include unsold vehicles regardless of parts status.

```

SELECT
  v.VIN,
  v.TypeName,
  v.MfgName AS Manufacturer,
  v.ModelName,
  v.ModelYear,
  v.FuelType,
  v.Drivetrain,
  GROUP_CONCAT(DISTINCT vc.Color ORDER BY vc.Color SEPARATOR ', ') AS Colors,
  v.Horsepower,
  ROUND(v.PurchPrice * 1.25 + SUM(CASE WHEN p.Status = 'Installed' THEN p.Quantity *
p.UnitPrice END) * 1.10, 2) AS SalePrice
FROM cs6400_sp26_team040.Vehicle v
LEFT JOIN cs6400_sp26_team040.VehicleColor vc
  ON v.VIN = vc.VIN
LEFT JOIN cs6400_sp26_team040.Sale s
  ON v.VIN = s.VIN
LEFT JOIN cs6400_sp26_team040.PartsOrder po
  ON v.VIN = po.VIN
LEFT JOIN cs6400_sp26_team040.Part p
  ON po.VIN = p.VIN AND po.OrderNum = p.OrderNum
LEFT JOIN cs6400_sp26_team040.Part pending_parts
  ON v.VIN = pending_parts.VIN AND pending_parts.Status != 'Installed'
LEFT JOIN cs6400_sp26_team040.VehicleColor color_filter
  ON v.VIN = color_filter.VIN AND color_filter.Color = '$Color'
WHERE
  ((s.VIN IS NULL AND '$StatusFilter' != 'Sold') OR (s.VIN IS NOT NULL AND
'$StatusFilter' IN ('Sold', 'All')) AND
  ('$UserRole' IN ('AcquisitionSpecialist', 'OperatingManager', 'Owner') OR pending_parts.VIN
IS NULL) AND
  ('$Color' = " OR color_filter.VIN IS NOT NULL) AND

```

```

('VIN' = " OR UPPER(v.VIN) = UPPER('$VIN')) AND
('$TypeName' = " OR v.TypeName = '$TypeName') AND
('$Manufacturer' = " OR v.MfgName = '$Manufacturer') AND
('$Year' = " OR v.ModelYear = '$Year') AND
('$FuelType' = " OR v.FuelType = '$FuelType') AND
('$Drivetrain' = " OR v.Drivetrain = '$Drivetrain') AND
('$Keyword' = " OR v.MfgName LIKE CONCAT('%', '$Keyword', '%') OR v.ModelName
LIKE CONCAT('%', '$Keyword', '%') OR CAST(v.ModelYear AS CHAR) LIKE CONCAT('%',
'$Keyword', '%') OR v.Notes LIKE CONCAT('%', '$Keyword', '%'))
GROUP BY v.VIN
ORDER BY v.VIN;

```

- If no vehicles match the criteria, display the message "Sorry, it looks like we don't have that in stock!".
- Else, display the matching results sorted by VIN in ascending order.
 - The user selects a specific vehicle to open the **Vehicle Detail form**.

User Login

Abstract Code + SQL

- User navigates to the **Login form** via the landing page.
- User enters their *username* (\$Username) and *password* (\$Password) into the input fields.
- When the ***Login button*** is clicked:
 - Run the **Login task**: Check the database to see if the *username* exists and the *password* matches the plaintext record.

```

SELECT Password FROM cs6400_sp26_team040.`User` WHERE Username =
'$Username';

```

- If User record is found but User.password != '\$Password':
 - Go back to **Login form** and display the error message: "Invalid username or password."
- Else if User record not found:
 - Go back to **Login form** and display the error message: "Invalid username or password".
- Else if User.password == '\$Password':
 - Store login information as session variable '\$UserID'.

```

SELECT Username FROM cs6400_sp26_team040.AcquisitionSpecialist WHERE
Username= '$Username';

```

```

SELECT Username FROM cs6400_sp26_team040.SalesAgent WHERE Username=

```

```
'$Username';
```

```
SELECT Username FROM cs6400_sp26_team040.OperatingManager WHERE  
Username= '$Username';
```

- Store the user's role and identity in the session, and redirect the user to the privileged **Search form**.
- Else username and password are empty, display **Login form**, with error message: “Please enter username and password”.

Add Vehicle

Abstract Code + SQL

- User clicks the *Add Vehicle button*
- Navigate to the **Add Vehicle form**.
- Run the **Lookup Customer task**: User enters a *SSN (\$SSN)* or *TaxID (\$TaxID)* and clicks the *Search Customer button*.

```
SELECT  
    c.CustomerID,  
    i.FirstName,  
    i.LastName  
FROM cs6400_sp26_team040.Customer c  
INNER JOIN cs6400_sp26_team040.Individual i  
    ON c.CustomerID = i.CustomerID  
WHERE i.SSN = '$SSN';
```

```
SELECT  
    c.CustomerID,  
    b.BusinessName  
FROM cs6400_sp26_team040.Customer c  
INNER JOIN cs6400_sp26_team040.Business b  
    ON c.CustomerID = b.CustomerID  
WHERE b.TaxID = '$TaxID';
```

- If customer found, select as Seller and store **\$CustomerID**
- If not found, user enters customer type (individual or business) and appropriate fields:
 - Individual: *first name (\$FirstName)*, *last name (\$LastName)*, *SSN (\$SSN)*, *phone (\$Phone)*, *address (\$StreetAddress)*, *email (\$Email)* (optional)
 - Business: *tax ID (\$TaxID)*, *business name (\$BusinessName)*, *contact title (\$ContactTitle)*, *contact first name (\$ContactFirstName)*, *contact last name (\$ContactLastName)*, *\$Phone*, *\$StreetAddress*, *\$Email* (optional)

```
INSERT INTO cs6400_sp26_team040.Customer (Phone, Email, StreetAddress, City, State,
PostalCode)
VALUES ('$Phone', '$Email', '$StreetAddress', '$City', '$State', '$PostalCode')
```

```
INSERT INTO cs6400_sp26_team040.Individual (FirstName, LastName, SSN, CustomerID)
VALUES ('$FirstName', '$LastName', '$SSN', '$CustomerID')
```

```
INSERT INTO cs6400_sp26_team040.Business (TaxID, BusinessName, ContactTitle,
ContactFirstName, ContactLastName, CustomerID)
VALUES ('$TaxID', '$BusinessName', '$ContactTitle', '$ContactFirstName',
'$ContactLastName', '$CustomerID')
```

- User fills in Vehicle details: *VIN* (\$VIN), *Type* (\$TypeName), *Manufacturer* (\$MfgName), *Model* (\$ModelName), *Model Year* (\$ModelYear), *Fuel Type* (\$FuelType), *Drivetrain* (\$Drivetrain), *Horsepower* (\$Horsepower), *Condition* (\$Condition), *Purchase Price* (\$PurchPrice), *Purchase Date* (\$PurchDate), *Notes* (\$Notes) (optional).

```
INSERT INTO cs6400_sp26_team040.Vehicle (VIN, TypeName, ModelName, ModelYear,
FuelType, Condition, Horsepower, Drivetrain, Notes, PurchPrice, PurchDate, TypeName,
MfgName)
VALUES ('$VIN', '$TypeName', '$ModelName', '$ModelYear', '$FuelType', '$Condition',
'$Horsepower', '$Drivetrain', '$Notes', '$PurchPrice', '$PurchDate', '$TypeName', '$MfgName');
```

- When the **Save Vehicle button** is clicked:
 - Validate *Model Year* is exactly four digits and does not exceed current year plus one.
 - If validation fails, return to **Add Vehicle form** and display error message.
 - If validation passes, create Customer record if new, then insert Vehicle record linked to Seller and Acquisition Specialist.
 - Redirect to **Vehicle Detail form** for the newly added Vehicle.

Add Parts Order

Abstract Code + SQL

- User clicks **"Add Parts Order" button** from **Vehicle Detail form**.
- Navigate to **Add Parts Order form**.
- Run **Search/Add Vendor task**: user enters Vendor Name (\$VendorName) and clicks Search.

```
SELECT
    VendorName,
    Phone,
    StreetAddress,
    City,
```

```

State,
PostalCode
FROM cs6400_sp26_team040.Vendor
WHERE VendorName = '$VendorName'

```

- When Add Vendor button is clicked:

```

INSERT INTO cs6400_sp26_team040.Vendor (VendorName, Phone, StreetAddress, City,
State, PostalCode)
VALUES ('$VendorName', '$Phone', '$StreetAddress', '$City', '$State', '$PostalCode');

```

- User enters one or more parts, for each part: *Description* (\$Description), *Vendor Part Number* (\$PartNumber), *Quantity* (\$Quantity), *Unit Price* (\$UnitPrice), *Status* (\$Status) (ordered, received, or installed).
- When Save Parts Order is clicked, generate parts order number by combining *VIN* (\$VIN) with ordinal of the order padded to three digits (e.g. VIN-001, VIN-002).
- Insert parts order and all parts records linked to vehicle and vendor.

```

INSERT INTO cs6400_sp26_team040.PartsOrder (VIN, OrderNum, VendorName)
VALUES ('$VIN', '$OrderNum', '$VendorName')

```

```

INSERT INTO cs6400_sp26_team040.Part (VIN, OrderNum, PartNumber, Description,
UnitPrice, Quantity, Status)
VALUES ('$VIN', '$OrderNum', '$PartNumber', '$Description', '$UnitPrice', '$Quantity',
'$Status')

```

- Redirect to **Vehicle Detail form**.

Sell Vehicle

Abstract Code + SQL

- User views an eligible vehicle on the **Vehicle Detail form** and clicks the ***Sell Vehicle button***.
- Navigate to the **Sell Vehicle form**.
- Run the **Lookup Customer task**: User enters the buyer's *SSN* or *TaxID* into the search field and clicks the ***Search button***.

```

SELECT Customer.CustomerID, Individual.FirstName, Individual.LastName FROM
cs6400_sp26_team040.Individual JOIN cs6400_sp26_team040.Customer ON
Individual.CustomerID = Customer.CustomerID WHERE Individual.SSN = '$SSN';

```

```

SELECT Customer.CustomerID, Business.BusinessName FROM
cs6400_sp26_team040.Business JOIN cs6400_sp26_team040.Customer ON
Business.CustomerID = Customer.CustomerID WHERE Business.TaxID = '$TaxID';

```

- If the buyer does not exist, the user enters the necessary contact details to create a new profile.

```
INSERT INTO cs6400_sp26_team040.Customer (Phone, Email, StreetAddress, City, State,
PostalCode) VALUES ('$Phone', '$Email', '$StreetAddress', '$City', '$State', '$PostalCode');
```

```
INSERT INTO cs6400_sp26_team040.Individual (SSN, FirstName, LastName, CustomerID)
VALUES ('$SSN', '$FirstName', '$LastName', LAST_INSERT_ID());
```

```
INSERT INTO cs6400_sp26_team040.Business (TaxID, BusinessName, ContactFirstName,
ContactLastName, ContactTitle, CustomerID) VALUES ('$TaxID', '$BusinessName',
'$ContactFirstName', '$ContactLastName', '$ContactTitle', LAST_INSERT_ID());
```

- When the **Confirm Sale button** is clicked:
 - Run the **Save Sale task**: Update the vehicle's record with the current *sale date*, assign the *buyer ID*, and lock the dynamically calculated sale price.

```
INSERT INTO cs6400_sp26_team040.Sale (VIN, Buyer_CustomerID,
SalesAgent_Username, SaleDate) VALUES ('$VIN', '$Buyer_CustomerID',
'$SalesAgent_Username', CURDATE());
```

- Return the user to the updated **Vehicle Detail form**

View Vehicle Detail

Abstract Code

- User selects a vehicle from the Search Results.
- Query vehicle record by *VIN*; display: *VIN, Type, Condition, Manufacturer, Model, Year, Fuel Type, Drivetrain, Color(s), Horsepower, Sale Price, Notes*.
- If user is Acquisition Specialist:
 - Also display: *Purchase Price, Purchase Date, Total Parts Cost*.

SQL Vehicle Detail

```
SELECT
    v.VIN,
    v.TypeName,
    v.MfgName,
    v.ModelName,
    v.ModelYear,
    v.FuelType,
    v.Drivetrain,
    v.Horsepower,
    v.Condition,
    GROUP_CONCAT(DISTINCT vc.Color ORDER BY vc.Color SEPARATOR ', ')
AS Colors,
    v.Notes,
    v.PurchPrice AS OriginalPurchasePrice,
    v.PurchDate AS PurchaseDate,
```

```

        IFNULL(SUM(CASE WHEN p.Status = 'Installed' THEN (p.UnitPrice *
p.Quantity) END), 0) AS TotalPartsCost,
        ROUND(v.PurchPrice * 1.25 + SUM(CASE WHEN p.Status = 'Installed' THEN
(p.UnitPrice * p.Quantity) END) * 1.10, 2) AS SalePrice
FROM cs6400_sp26_team040.Vehicle v
LEFT JOIN cs6400_sp26_team040.VehicleColor vc
    ON v.VIN = vc.VIN
LEFT JOIN cs6400_sp26_team040.PartsOrder po
    ON v.VIN = po.VIN
LEFT JOIN cs6400_sp26_team040.Part p
    ON po.VIN = p.VIN AND po.OrderNum = p.OrderNum
WHERE v.VIN = '$VIN'
GROUP BY v.VIN;

```

- Display parts list with: part number, description, vendor, parts order number, unit price, quantity, status.

SQL Parts List

```

SELECT
    p.PartNumber,
    p.Description,
    po.VendorName AS Vendor,
    po.OrderNum AS PartsOrder,
    p.UnitPrice,
    p.Quantity,
    p.Status
FROM cs6400_sp26_team040.Part p
INNER JOIN cs6400_sp26_team040.PartsOrder po
    ON p.VIN = po.VIN AND p.OrderNum = po.OrderNum
WHERE p.VIN = '$VIN';

```

- For each part not yet installed, provide status update control (ordered→received, received→installed, ordered→installed only; no reversals).
- Provide **"Add Parts Order" button** linking to **Add Parts Order form**.
- If user is Sales Agent:

SQL Vehicle Detail

```

SELECT
    v.VIN,
    v.TypeName,
    v.MfgName,
    v.ModelName,
    v.ModelYear,
    v.FuelType,
    v.Drivetrain,
    v.Horsepower,
    v.Condition,
    GROUP_CONCAT(DISTINCT vc.Color ORDER BY vc.Color SEPARATOR ', ')

```

```

AS Colors,
    v.Notes,
    v.PurchDate AS PurchaseDate,
    ROUND(v.PurchPrice * 1.25 + SUM(CASE WHEN p.Status = 'Installed' THEN
(p.UnitPrice * p.Quantity) END) * 1.10, 2) AS SalePrice
FROM cs6400_sp26_team040.Vehicle v
LEFT JOIN cs6400_sp26_team040.VehicleColor vc
    ON v.VIN = vc.VIN
LEFT JOIN cs6400_sp26_team040.PartsOrder po
    ON v.VIN = po.VIN
LEFT JOIN cs6400_sp26_team040.Part p
    ON po.VIN = p.VIN AND po.OrderNum = p.OrderNum AND p.Status =
'Installed'
WHERE v.VIN = '$VIN'
GROUP BY v.VIN;

```

- If vehicle has no pending parts and has not been sold, display ***"Sell Vehicle" button*** linking to **Sell Vehicle form**.
- If user is Operating Manager:
 - The Operating Manager has view-only access to all of the information below.
 - Also display: *Purchase Price, Purchase Date, Total Parts Cost*.
 - Display seller contact info (excluding SSN/Tax ID), Acquisition Specialist's *First Name* and *Last Name*.
 - If individual: *First Name, Last Name, Phone, Address, Email*
 - If business: *Business Name, Contact Title, Contact First Name, Contact Last Name, Phone, Address, Email*
 - Display parts list (same fields as above, view only).
 - If vehicle has been sold, also display: *Buyer Contact Info* (excluding SSN/Tax ID), *Sale Date*, Sales Agent's *First Name* and *Last Name*.

SQL Vehicle Detail

```

SELECT
    v.VIN,
    v.TypeName,
    v.MfgName,
    v.ModelName,
    v.ModelYear,
    v.FuelType,
    v.Drivetrain,
    v.Horsepower,
    v.Condition,
    GROUP_CONCAT(DISTINCT vc.Color ORDER BY vc.Color SEPARATOR ', ')
AS Colors,
    v.Notes,
    v.PurchPrice AS OriginalPurchasePrice,
    v.PurchDate AS PurchaseDate,
    SUM(CASE WHEN p.Status = 'Installed' THEN p.UnitPrice * p.Quantity END) AS
TotalPartsCost,

```

```

ROUND(v.PurchPrice * 1.25 + SUM(CASE WHEN p.Status = 'Installed' THEN
p.UnitPrice * p.Quantity END) * 1.10, 2) AS SalePrice,
MAX(acq_user.FirstName) AS AcqSpecialistFirstName,
MAX(acq_user.LastName) AS AcqSpecialistLastName,
MAX(seller_c.Phone) AS SellerPhone,
MAX(seller_c.Email) AS SellerEmail,
MAX(seller_c.StreetAddress) AS SellerStreet,
MAX(seller_c.City) AS SellerCity,
MAX(seller_c.State) AS SellerState,
MAX(seller_c.PostalCode) AS SellerPostal,
COALESCE(CONCAT(MAX(seller_i.FirstName), ' ', MAX(seller_i.LastName)),
MAX(seller_b.BusinessName)) AS SellerName,
MAX(seller_b.ContactFirstName) AS SellerContactFirstName,
MAX(seller_b.ContactLastName) AS SellerContactLastName,
MAX(seller_b.ContactTitle) AS SellerContactTitle,
MAX(s.SaleDate) AS SaleDate,
MAX(sales_user.FirstName) AS SalesAgentFirstName,
MAX(sales_user.LastName) AS SalesAgentLastName,
MAX(buyer_c.Phone) AS BuyerPhone,
MAX(buyer_c.Email) AS BuyerEmail,
MAX(buyer_c.StreetAddress) AS BuyerStreet,
MAX(buyer_c.City) AS BuyerCity,
MAX(buyer_c.State) AS BuyerState,
MAX(buyer_c.PostalCode) AS BuyerPostal,
COALESCE(CONCAT(MAX(buyer_i.FirstName), ' ', MAX(buyer_i.LastName)),
MAX(buyer_b.BusinessName)) AS BuyerName,
MAX(buyer_b.ContactFirstName) AS BuyerContactFirstName,
MAX(buyer_b.ContactLastName) AS BuyerContactLastName,
MAX(buyer_b.ContactTitle) AS BuyerContactTitle
FROM cs6400_sp26_team040.Vehicle v
LEFT JOIN cs6400_sp26_team040.VehicleColor vc
ON v.VIN = vc.VIN
LEFT JOIN cs6400_sp26_team040.PartsOrder po
ON v.VIN = po.VIN
LEFT JOIN cs6400_sp26_team040.Part p
ON po.VIN = p.VIN AND po.OrderNum = p.OrderNum
INNER JOIN cs6400_sp26_team040.`User` acq_user
ON v.AcqSpecialist_Username = acq_user.Username
INNER JOIN cs6400_sp26_team040.Customer seller_c
ON v.Purchaser_CustomerID = seller_c.CustomerID
LEFT JOIN cs6400_sp26_team040.Individual seller_i
ON seller_c.CustomerID = seller_i.CustomerID
LEFT JOIN cs6400_sp26_team040.Business seller_b
ON seller_c.CustomerID = seller_b.CustomerID
LEFT JOIN cs6400_sp26_team040.Sale s
ON v.VIN = s.VIN
LEFT JOIN cs6400_sp26_team040.`User` sales_user
ON s.SalesAgent_Username = sales_user.Username
LEFT JOIN cs6400_sp26_team040.Customer buyer_c
ON s.Buyer_CustomerID = buyer_c.CustomerID

```

```

LEFT JOIN cs6400_sp26_team040.Individual buyer_i
  ON buyer_c.CustomerID = buyer_i.CustomerID
LEFT JOIN cs6400_sp26_team040.Business buyer_b
  ON buyer_c.CustomerID = buyer_b.CustomerID
WHERE v.VIN = '$VIN'
GROUP BY v.VIN;

```

SQL Parts List

```

SELECT
  p.PartNumber,
  p.Description,
  po.VendorName AS Vendor,
  po.OrderNum AS PartsOrder,
  p.UnitPrice,
  p.Quantity,
  p.Status
FROM cs6400_sp26_team040.Part p
INNER JOIN cs6400_sp26_team040.PartsOrder po
  ON p.VIN = po.VIN AND p.OrderNum = po.OrderNum
WHERE p.VIN = '$VIN';

```

- If user is Owner:
 - Access to complete functionality as described above.

SQL Vehicle Detail

```

SELECT
  v.VIN,
  v.TypeName,
  v.MfgName,
  v.ModelName,
  v.ModelYear,
  v.FuelType,
  v.Drivetrain,
  v.Horsepower,
  v.Condition,
  GROUP_CONCAT(DISTINCT vc.Color ORDER BY vc.Color SEPARATOR ', ')
AS Colors,
  v.Notes,
  v.PurchPrice AS OriginalPurchasePrice,
  v.PurchDate AS PurchaseDate,
  SUM(CASE WHEN p.Status = 'Installed' THEN p.UnitPrice * p.Quantity END) AS
TotalPartsCost,
  ROUND(v.PurchPrice * 1.25 + SUM(CASE WHEN p.Status = 'Installed' THEN
p.UnitPrice * p.Quantity END) * 1.10, 2) AS SalePrice,
  MAX(acq_user.FirstName) AS AcqSpecialistFirstName,
  MAX(acq_user.LastName) AS AcqSpecialistLastName,
  MAX(seller_c.Phone) AS SellerPhone,
  MAX(seller_c.Email) AS SellerEmail,

```

```

MAX(seller_c.StreetAddress) AS SellerStreet,
MAX(seller_c.City) AS SellerCity,
MAX(seller_c.State) AS SellerState,
MAX(seller_c.PostalCode) AS SellerPostal,
COALESCE(CONCAT(MAX(seller_i.FirstName), ' ', MAX(seller_i.LastName)),
MAX(seller_b.BusinessName)) AS SellerName,
MAX(seller_b.ContactFirstName) AS SellerContactFirstName,
MAX(seller_b.ContactLastName) AS SellerContactLastName,
MAX(seller_b.ContactTitle) AS SellerContactTitle,
MAX(s.SaleDate) AS SaleDate,
MAX(sales_user.FirstName) AS SalesAgentFirstName,
MAX(sales_user.LastName) AS SalesAgentLastName,
MAX(buyer_c.Phone) AS BuyerPhone,
MAX(buyer_c.Email) AS BuyerEmail,
MAX(buyer_c.StreetAddress) AS BuyerStreet,
MAX(buyer_c.City) AS BuyerCity,
MAX(buyer_c.State) AS BuyerState,
MAX(buyer_c.PostalCode) AS BuyerPostal,
COALESCE(CONCAT(MAX(buyer_i.FirstName), ' ', MAX(buyer_i.LastName)),
MAX(buyer_b.BusinessName)) AS BuyerName,
MAX(buyer_b.ContactFirstName) AS BuyerContactFirstName,
MAX(buyer_b.ContactLastName) AS BuyerContactLastName,
MAX(buyer_b.ContactTitle) AS BuyerContactTitle
FROM cs6400_sp26_team040.Vehicle v
LEFT JOIN cs6400_sp26_team040.VehicleColor vc
    ON v.VIN = vc.VIN
LEFT JOIN cs6400_sp26_team040.PartsOrder po
    ON v.VIN = po.VIN
LEFT JOIN cs6400_sp26_team040.Part p
    ON po.VIN = p.VIN AND po.OrderNum = p.OrderNum
INNER JOIN cs6400_sp26_team040.`User` acq_user
    ON v.AcqSpecialist_Username = acq_user.Username
INNER JOIN cs6400_sp26_team040.Customer seller_c
    ON v.Purchaser_CustomerID = seller_c.CustomerID
LEFT JOIN cs6400_sp26_team040.Individual seller_i
    ON seller_c.CustomerID = seller_i.CustomerID
LEFT JOIN cs6400_sp26_team040.Business seller_b
    ON seller_c.CustomerID = seller_b.CustomerID
LEFT JOIN cs6400_sp26_team040.Sale s
    ON v.VIN = s.VIN
LEFT JOIN cs6400_sp26_team040.`User` sales_user
    ON s.SalesAgent_Username = sales_user.Username
LEFT JOIN cs6400_sp26_team040.Customer buyer_c
    ON s.Buyer_CustomerID = buyer_c.CustomerID
LEFT JOIN cs6400_sp26_team040.Individual buyer_i
    ON buyer_c.CustomerID = buyer_i.CustomerID
LEFT JOIN cs6400_sp26_team040.Business buyer_b
    ON buyer_c.CustomerID = buyer_b.CustomerID
WHERE v.VIN = '$VIN'
GROUP BY v.VIN;

```

SQL Parts List

```
SELECT
    p.PartNumber,
    p.Description,
    po.VendorName AS Vendor,
    po.OrderNum AS PartsOrder,
    p.UnitPrice,
    p.Quantity,
    p.Status
FROM cs6400_sp26_team040.Part p
INNER JOIN cs6400_sp26_team040.PartsOrder po
    ON p.VIN = po.VIN AND p.OrderNum = po.OrderNum
WHERE p.VIN = '$VIN';
```

Parts Statistics

Abstract Code

- User clicked on **Parts Statistics Report** from the **Reports dropdown**
- Query for information on the Vendor, sum of UnitPrice (quantity * UnitPrice) and sum of Quantity
 - From a merged table of Vendor, PartsOrder and Parts
 - That is grouped by and sorted alphabetically on VendorName
- Render and output the report.

SQL

```
SELECT
    po.VendorName,
    SUM(p.Quantity) AS TotalQuantity,
    SUM(p.Quantity * p.UnitPrice) AS TotalSpent
FROM cs6400_sp26_team040.Vendor v
LEFT JOIN cs6400_sp26_team040.PartsOrder po
    ON v.VendorName = po.VendorName
LEFT JOIN cs6400_sp26_team040.Part p
    ON po.VIN = p.VIN AND po.OrderNum = p.OrderNum
GROUP BY v.VendorName
ORDER BY v.VendorName;
```

Price Per Conditions

Abstract Code

- User clicked on **Price Per Conditions Report** from the **Reports dropdown**
- Get all unique values of Conditions and TypeName from the Vehicles table.

- Initialize a matrix of unique Conditions and TypeName with a default of \$0.
- Query TypeName, Conditions, and average of PurchPrice from the Vehicles Table
 - Group by the TypeName and Conditions
- Populate the matrix with the queried values.
- Render and output the report.

SQL

```

WITH cond AS (
    SELECT 'Excellent' AS Condition, 1 AS SortOrder
    UNION SELECT 'Very Good', 2
    UNION SELECT 'Good', 3
    UNION SELECT 'Fair', 4
    UNION SELECT 'Rough', 5
)
SELECT
    cond.Condition,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'Convertible' THEN
v.PurchPrice END), 0), 2)) AS Convertible,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'Coupe' THEN v.PurchPrice
END), 0), 2)) AS Coupe,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'Minivan' THEN
v.PurchPrice END), 0), 2)) AS Minivan,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'Sedan' THEN v.PurchPrice
END), 0), 2)) AS Sedan,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'SUV' THEN v.PurchPrice
END), 0), 2)) AS SUV,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'Truck' THEN v.PurchPrice
END), 0), 2)) AS Truck,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'Van' THEN v.PurchPrice
END), 0), 2)) AS Van,
    CONCAT('$', ROUND(IFNULL(AVG(CASE WHEN v.TypeName = 'Other' THEN v.PurchPrice
END), 0), 2)) AS Other
FROM cond
LEFT JOIN cs6400_sp26_team040.Vehicle v
    ON cond.Condition = v.Condition
GROUP BY cond.Condition, cond.SortOrder
ORDER BY cond.SortOrder;

```

Monthly Sales Summary

Abstract Code

- User clicked on **Monthly Sales Summary Report** from the **Monthly Sales Summary Report**
- Query month and year (from SaleDate), gross sales income, count of VIN (as total vehicles sold), and total net income.

- From the Vehicles table merged with Parts.
 - Group and order by month, year (descending; most recent month and year first).
- Calculate the gross sales income as the sum of (PurchPrice * 1.25) plus the sum of parts costs ((UnitPrice * Quantity) * 1.1).
- Subtract the sum of PurchPrice and parts costs from gross sales income as net income.
- Drop every column except for month, year, total vehicles sold, gross sales income and net income.
- Render and output the report.

SQL

```
WITH parts_costs AS (
  SELECT
    VIN,
    SUM(UnitPrice * Quantity) AS TotalCost
  FROM cs6400_sp26_team040.Part
  WHERE Status = 'Installed'
  GROUP BY VIN
)
SELECT
  YEAR(s.SaleDate) AS SaleYear,
  MONTH(s.SaleDate) AS SaleMonth,
  COUNT(s.VIN) AS TotalVehiclesSold,
  ROUND(SUM(v.PurchPrice * 1.25 + parts_costs.TotalCost * 1.10), 2) AS GrossSalesIncome,
  ROUND(SUM((v.PurchPrice * 1.25 + parts_costs.TotalCost * 1.10) - v.PurchPrice -
  parts_costs.TotalCost), 2) AS TotalNetIncome
FROM cs6400_sp26_team040.Sale s
INNER JOIN cs6400_sp26_team040.Vehicle v
  ON s.VIN = v.VIN
LEFT JOIN parts_costs
  ON v.VIN = parts_costs.VIN
GROUP BY YEAR(s.SaleDate), MONTH(s.SaleDate)
ORDER BY SaleYear DESC, SaleMonth DESC;
```

Monthly Sales Drilldown

Abstract Code + SQL

- User clicked on a valid *month/year* (\$Month, \$Year) from the *Monthly Sales Summary Report*
- Query FirstName and LastName, count of VIN (as total vehicles sold), sum of PurchPrice * 1.25 plus sum of UnitPrice * 1.1 * Quantity (as total sales)
 - From the Vehicles table merged with Parts.
 - Filter to Vehicles that have a SaleDate for that month/year
 - Order by total vehicles sold, or total sales descending.

```
WITH parts_costs AS (
```

```

SELECT
    p.VIN,
    SUM(p.UnitPrice * p.Quantity) AS TotalCost
FROM cs6400_sp26_team040.Part p
WHERE p.Status = 'Installed'
GROUP BY p.VIN
)
SELECT
    u.FirstName,
    u.LastName,
    COUNT(DISTINCT s.VIN) AS TotalVehiclesSold,
    SUM(ROUND(v.PurchPrice * 1.25 + parts_costs.TotalCost * 1.10, 2)) AS TotalSales
FROM cs6400_sp26_team040.Sale s
INNER JOIN cs6400_sp26_team040.Vehicle v
    ON s.VIN = v.VIN
INNER JOIN cs6400_sp26_team040.SalesAgent sa
    ON s.SalesAgent_Username = sa.Username
INNER JOIN cs6400_sp26_team040.`User` u
    ON sa.Username = u.Username
LEFT JOIN parts_costs
    ON v.VIN = parts_costs.VIN
WHERE YEAR(s.SaleDate) = '$Year'
    AND MONTH(s.SaleDate) = '$Month'
GROUP BY u.Username, u.FirstName, u.LastName
ORDER BY TotalVehiclesSold DESC, TotalSales DESC;

```

- Render and output the report.

Seller History

Abstract Code

- User clicked on ***Seller History Report*** from the ***Reports dropdown***
- Query FirstName and LastName (if individual) or BusinessName (if business) and set as Seller, count of VIN (as vehicles purchased), average of total Quantity and average of total UnitPrice
 - From the Individuals, Business Tables, Vehicles, and Parts.
 - Filter to Vehicles that have a SaleDate for that month/year
 - Group by Seller. Order by count of VIN (descending), and average PurchPrice (ascending).
- For each Seller with ≥ 5 average total Quantity OR $> \$500$ for average total UnitPrice, highlight with red.
- Render and output the report.

SQL

```

SELECT
    COALESCE(CONCAT(i.FirstName, ' ', i.LastName), b.BusinessName) AS SellerName,

```

```

COUNT(DISTINCT v.VIN) AS TotalVehicles, ROUND(AVG(v.PurchPrice), 2) AS
AvgPurchasePrice,
SUM(p.Quantity) / COUNT(DISTINCT v.VIN) AS AvgPartsQuantity,
ROUND(SUM(p.Quantity * p.UnitPrice) / COUNT(DISTINCT v.VIN), 2) AS AvgPartsCost
FROM cs6400_sp26_team040.Vehicle v
INNER JOIN cs6400_sp26_team040.Customer c
    ON v.Purchaser_CustomerID = c.CustomerID
LEFT JOIN cs6400_sp26_team040.Individual i
    ON c.CustomerID = i.CustomerID
LEFT JOIN cs6400_sp26_team040.Business b
    ON c.CustomerID = b.CustomerID
LEFT JOIN cs6400_sp26_team040.PartsOrder po
    ON v.VIN = po.VIN
LEFT JOIN cs6400_sp26_team040.Part p
    ON po.VIN = p.VIN AND po.OrderNum = p.OrderNum
GROUP BY c.CustomerID, i.FirstName, i.LastName, b.BusinessName
ORDER BY TotalVehicles DESC, AvgPurchasePrice;

```

Average Time in Inventory

Abstract Code

- User clicked on *Average Time in Inventory Report* from the *Reports dropdown*
- Query TypeName, average of SaleDate - PurchDate + 1 (as average days in inventory)
 - From the Vehicles table.
 - Filter to exclude TypeNames with no SaleDate.
 - Group by TypeName.
- Merge with a table of unique TypeNames, where non-matches get assigned “N/A”
- Render and output the report.

SQL

```

SELECT
    vt.TypeName,
    CASE
        WHEN COUNT(s.VIN) = 0 THEN 'N/A'
        ELSE AVG(DATEDIFF(s.SaleDate, v.PurchDate) + 1)
    END AS AvgDaysInInventory
FROM cs6400_sp26_team040.VehicleType vt
LEFT JOIN cs6400_sp26_team040.Vehicle v
    ON vt.TypeName = v.TypeName
LEFT JOIN cs6400_sp26_team040.Sale s
    ON v.VIN = s.VIN
GROUP BY vt.TypeName
ORDER BY vt.TypeName;

```

Search/Add Vendor

Abstract Code

- User enters *vendor name* (\$VendorName) and clicks Search.
- Query database for exact vendor name match.

```
SELECT VendorName FROM cs6400_sp26_team040.Vendor WHERE VendorName = '$VendorName'
```

- If vendor found, select as vendor for the parts order.
- If not found, user enters vendor details: street address, city, state, postal code, phone number.
- Create new vendor record and select as vendor for the parts order.

```
INSERT INTO cs6400_sp26_team040.Vendor (VendorName, Phone, StreetAddress, City, State, PostalCode)  
VALUES ('$VendorName', '$Phone', '$StreetAddress', '$City', '$State', '$PostalCode');
```

Update Part Status

Abstract Code + SQL

- User selects a new status for a part from the **Vehicle Detail form**.
- Validate transition is forward-only: ordered→received, received→installed, or ordered→installed.
- If invalid, display error message.
- If valid, update part status record in database.

```
SELECT Status FROM cs6400_sp26_team040.Part WHERE VIN = '$VIN' AND OrderNum = '$OrderNum' AND PartNumber = '$PartNumber';
```

```
UPDATE cs6400_sp26_team040.Part SET Status = '$NewStatus' WHERE VIN = '$VIN' AND OrderNum = '$OrderNum' AND PartNumber = '$PartNumber';
```

- Refresh parts list on **Vehicle Detail form**.